Air University



Parallel and Distributed Computing (Lab)

Year 2026

Mr. Qaiser Ali

Academic Year: 2024 – 2028

Department of Cyber Security

USAMA ARSHAD | 247141

BS – Cyber Security – Fall – 24 (A)

Date of Submission: September 23, 2026

Lab Task # 01

Question 01:

Illustrate the concept of variable hiding in Java by writing a program that defines a superclass P with an instance variable int a = 30 and a child class Q that inherits from P with a shadowed variable int a = 50. Instantiate class Q using a reference variable q, assign q to a superclass reference variable p, display the values accessed via both q.a and p.a, and explain how compile-time binding governs instance variable access compared to dynamic method overriding.

Code:

```
// Superclass
class P {
    int a = 30;
}

// Child class inheriting from P
class Q extends P {
    int a = 50; // Hides P.a
}

public class VariableHidingDemo {
    public static void main(String[] args) {
        // Instantiate Q using reference variable q of type Q
        Q q = new Q();

        // Assign q to a superclass reference variable p of type P
        P p = q;

        // Access instance variables through both references
        System.out.println("Value accessed via q.a: " + q.a);
        System.out.println("Value accessed via p.a: " + p.a);
    }
}
```

Output:

```
Value accessed via q.a: 50
Value accessed via p.a: 30
```

Explanation:

- We have a parent class P with int a = 30, and a child class Q extends P with another int a = 50. This is called variable hiding, not overriding.
- In main, Q q = new Q(); creates one Q object. Since Q inherits from P, the object has both P.a and Q.a inside it.
- P p = q; is allowed because a child object can be stored in a parent reference. But p is still a reference of type P.
- q.a accesses Q's own a, so it prints 50.
- p.a accesses P's a, so it prints 30, even though p points to a Q object. **Main point:** For instance variables, Java uses the reference type, not the object type. For methods, it would use the object type.

Question 02:

Illustrate the fundamental concept of single inheritance in Java by completing the following requirements:

1. **Construct a superclass named Animal that declares a String instance variable name and a void method named eat() that prints a message.**
2. **Construct a subclass named Dog that inherits from Animal and includes a method named display() to print a custom message using the inherited name variable.**
3. **Instantiate an object of class Dog in the main method, assign a value to the name variable, and invoke the methods of both the superclass and subclass to demonstrate how child classes inherit superclass members.**

Code:

```java
// Superclass
class Animal {
    // Instance variable accessible to subclasses
    String name;

    // Superclass method
    public void eat() {
        System.out.println(name + " is eating food.");
    }
}

// Subclass inheriting from Animal
class Dog extends Animal {
    // Subclass method utilizing the inherited field
    public void display() {
        System.out.println("Dog's Name: " + name);
    }
}

public class SingleInheritanceDemo {
    public static void main(String[] args) {
        // Instantiate an object of subclass Dog
        Dog myDog = new Dog();

        // Assign a value to the inherited instance variable
        myDog.name = "Tommy";

        // Invoke superclass method
        myDog.eat();

        // Invoke subclass method
        myDog.display();
    }
}
```

Output:

```
Tommy is eating food.
Dog's Name: Tommy
```

Explanation:

- Animal is the superclass with a variable name and a method eat(). Dog is the subclass that inherits from Animal using extends, so it automatically gets name and eat().
- Inside main, we create a Dog object called myDog. Since Dog inherits from Animal, we can directly set the inherited variable using myDog.name = "Tommy";.
- myDog.eat(); calls the inherited superclass method. It uses the inherited name value, so it prints: Tommy is eating food.
- myDog.display(); calls the subclass's own method, which also uses the inherited name variable, so it prints: 'Dog's Name: Tommy
- This is a simple example of single inheritance: one subclass (Dog) inherits from one superclass (Animal), and the subclass can reuse the superclass's fields and methods directly.

Question 03:

Design and implement a java program centered around an Employee class that encapsulates three private instance variables: firstName (String), lastName (String), and monthlySalary (int). Construct a parameterized constructor to initialize these attributes while ensuring that any non-positive salary value is automatically defaulted to zero, and supply public getter and setter methods for each attribute that maintain this same non-negative validation rule. Include a helper method within the class to compute the annual salary based on the monthly rate. Finally, write a driver class containing a main method that instantiates two distinct Employee objects, calculates and displays each employee's initial yearly salary, applies a ten percent raise to their monthly salaries using the appropriate setter methods, and displays their newly updated yearly salaries.

Code:

```
class Employee {
    // Private instance variables
    private String firstName;
    private String lastName;
    private int monthlySalary;

    // Parameterized Constructor
    public Employee(String firstName, String lastName, int monthlySalary) {
        this.firstName = firstName;
        this.lastName = lastName;
        // Apply validation directly or route via setter
        setMonthlySalary(monthlySalary);
    }

    // Getters and Setters
    public String getFirstName() {
        return firstName;
    }

    public void setFirstName(String firstName) {
        this.firstName = firstName;
    }

    public String getLastName() {
        return lastName;
    }
```

```
    public void setLastName(String lastName) {
        this.lastName = lastName;
    }

    public int getMonthlySalary() {
        return monthlySalary;
    }

    // Mutator with non-negative validation rule
    public void setMonthlySalary(int monthlySalary) {
        if (monthlySalary > 0) {
            this.monthlySalary = monthlySalary;
        } else {
            this.monthlySalary = 0;
        }
    }

    // Helper method to compute annual salary
    public int getAnnualSalary() {
        return this.monthlySalary * 12;
    }
}

public class EmployeeTest {
    public static void main(String[] args) {
        // Instantiate two Employee objects
        Employee emp1 = new Employee("Asad", "Farooq", 5000);
        Employee emp2 = new Employee("Ibtehaj", "Ali", -1200); // Invalid salary, should
default to 0

        // Display initial yearly salaries
        System.out.println("--- Initial Yearly Salaries ---");
        System.out.println(emp1.getFirstName() + " " + emp1.getLastName() +
                ": $" + emp1.getAnnualSalary());
        System.out.println(emp2.getFirstName() + " " + emp2.getLastName() +
                ": $" + emp2.getAnnualSalary());

        // Apply a 10% raise to both employees' monthly salaries
        emp1.setMonthlySalary((int) (emp1.getMonthlySalary() * 1.10));
        emp2.setMonthlySalary((int) (emp2.getMonthlySalary() * 1.10));

        // Display updated yearly salaries
        System.out.println("\n--- Yearly Salaries After 10% Raise ---");
        System.out.println(emp1.getFirstName() + " " + emp1.getLastName() +
                ": $" + emp1.getAnnualSalary());
```

```
        System.out.println(emp2.getFirstName() + " " + emp2.getLastName() +
                ": $" + emp2.getAnnualSalary());
    }
}
```

Output:

```
--- Initial Yearly Salaries ---
Asad Farooq: $60000
Ibtehaj Ali: $0

--- Yearly Salaries After 10% Raise ---
Asad Farooq: $66000
Ibtehaj Ali: $0
```

Explanation:

- Employee is a class that uses encapsulation: its fields (firstName, lastName, monthlySalary) are private, so they can only be accessed through public getters and setters.
- The constructor takes values and stores them. For salary, it calls setMonthlySalary(), which applies validation: if salary is positive, it is stored; if zero or negative, it becomes `0`.
- getAnnualSalary() is a helper method that simply multiplies the monthly salary by 12.
- In main, two employees are created: emp1 with a valid salary of 5000, and emp2 with an invalid salary of -1200, so emp2's salary becomes `0`.
- Then both employees get a 10% raise using setMonthlySalary() again, and their updated yearly salaries are printed. emp1 becomes 5500 per month → 66000 per year, while emp2 stays `0`.

Question 04:

You're working as a software developer at an e-commerce platform that is upgrading its checkout system to handle different fulfillment pipelines. To support this upgrade, your task is to build an extensible order-processing hierarchy in Java. Every order passing through the platform possesses a unique orderId and a basePrice, along with a specialized method to compute the final billable total. Standard customer purchases are processed as a StandardOrder, which calculates the total cost by applying a flat 5% shipping fee on top of the base price. Conversely, limited-time promotions trigger a FlashSaleOrder, which requires expedited handling; this order type first grants customers a 10% promotional discount on the basePrice before adding a flat $10 rush shipping fee. To implement this engineering solution, design the class hierarchy using object-oriented principles, leveraging the super keyword to pass shared attributes to the parent constructor and using method overriding so each subclass accurately computes its own final order cost.

Code:

```
// Base Class
class Order {
    protected String orderId;
    protected double basePrice;

    // Base Constructor
    public Order(String orderId, double basePrice) {
        this.orderId = orderId;
        this.basePrice = basePrice;
    }

    // Method to be overridden by subclasses
    public double calculateTotal() {
```

```
        return this.basePrice;
    }

    public void printSummary() {
        System.out.printf("Order ID: %s | Base Price: $%.2f | Final Billable Total: $%.2f%n",
                orderId, basePrice, calculateTotal());
    }
}

// Subclass 1: Standard Order (Flat 5% shipping fee)
class StandardOrder extends Order {

    public StandardOrder(String orderId, double basePrice) {
        // Forward shared attributes to parent constructor
        super(orderId, basePrice);
    }

    @Override
    public double calculateTotal() {
        double shippingFee = this.basePrice * 0.05;
        return this.basePrice + shippingFee;
    }
}

// Subclass 2: Flash Sale Order (10% discount on base price + $10 rush shipping)
class FlashSaleOrder extends Order {

    public FlashSaleOrder(String orderId, double basePrice) {
        // Forward shared attributes to parent constructor
        super(orderId, basePrice);
    }

    @Override
    public double calculateTotal() {
        double discountedPrice = this.basePrice * 0.90; // 10% discount
        double rushShippingFee = 10.00;
        return discountedPrice + rushShippingFee;
    }
}

// Driver class demonstrating polymorphism
public class CheckoutSystem {
    public static void main(String[] args) {
        // Using base class references to demonstrate runtime polymorphism
        Order standardOrder = new StandardOrder("ORD-STD-101", 100.00);
        Order flashSaleOrder = new FlashSaleOrder("ORD-FLS-202", 80.00);

        System.out.println("=== Checkout Pipeline Processing ===");
        standardOrder.printSummary();
        flashSaleOrder.printSummary();
    }
}
```

Output:

```
=== Checkout Pipeline Processing ===
Order ID: ORD-STD-101 | Base Price: $100.00 | Final Billable Total: $105.00
Order ID: ORD-FLS-202 | Base Price: $80.00 | Final Billable Total: $82.00
```

Explanation:

- Order is the parent class with shared fields (orderId, basePrice), a constructor, and two methods: calculateTotal() and printSummary(). The printSummary() method calls calculateTotal(), so the final total depends on which subclass object is running.
- StandardOrder and FlashSaleOrder are child classes that extend Order. Both use super(orderId, basePrice) to pass shared data to the parent constructor.
- Each child class overrides calculateTotal():
  - → StandardOrder adds a 5% shipping fee.
  - → FlashSaleOrder gives a 10% discount and adds $10 rush shipping.
- In main, we create child objects but store them in parent references (Order standardOrder = new StandardOrder(...)). This is upcasting, and it is allowed because a child object is also an Order.
- When printSummary() runs, Java uses runtime polymorphism (dynamic method dispatch). Even though the reference type is Order, the actual object type decides which calculateTotal() runs. So:
  - → Standard order: $100 + 5% = $105.00
  - → Flash sale order: $80 - 10% + $10 = $82.00

- End of Lab Task -